

Code Explanation

SCD Type 2 Updater Test Explanation

This test class is designed to verify the functionality of the `update\_scd\_type2` function, which is used to update a Slowly Changing Dimension (SCD) Type 2 table. The test creates a SparkSession, sets up temporary Delta tables, and tests the update functionality.

Overview of the Code

The code is a unit test written in Python using the unittest framework. It tests the `update\_scd\_type2` function, which updates an SCD Type 2 table based on changes in a customer table. The test creates a SparkSession, sets up temporary Delta tables for customer and order summary data, updates the customer table, and then checks the order summary table for the expected changes.

Step 1: Setting up the SparkSession and Temporary Directory

This step creates a SparkSession with the necessary configurations for Delta Lake and sets up a temporary directory for storing Delta tables.

```
cls.spark = SparkSession.builder
    .appName("TestScdType2Updater")
    .master("local[1]")
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
    .getOrCreate()

# Create temporary directory for Delta tables
cls.temp_dir = tempfile.mkdtemp()
```

Step 2: Creating Sample Customer and Order Summary Data

This step creates sample data for the customer and order summary tables, including initial and updated customer data.

```
customer_data_initial = [
    ("C001", "John Doe", "john@example.com", "North"),
    ("C002", "Jane Smith", "jane@example.com", "South"),
    ("C003", "Bob Johnson", "bob@example.com", "East")
]

customer_schema = StructType([
    StructField("CustId", StringType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])

cls.customer_df_initial = cls.spark.createDataFrame(customer_data_initial, schema=customer_schema)

# Create sample customer data (updated)
customer_data_updated = [
    ("C001", "John Doe Updated", "john_new@example.com", "North"),
```

```

        ("C002", "Jane Smith", "jane@example.com", "West"),
        ("C003", "Bob Johnson", "bob@example.com", "East")
    ]

cls.customer_df_updated = cls.spark.createDataFrame(customer_data_updated, schema=customer_schema)

```

Step 3: Creating Delta Tables for Customer and Order Summary Data

This step creates Delta tables for the customer and order summary data using the sample data created in the previous step.

```

cls.customer_path = os.path.join(cls.temp_dir, "customer")
cls.ordersummary_path = os.path.join(cls.temp_dir, "ordersummary")

cls.customer_df_initial.write.format("delta").save(cls.customer_path)
cls.ordersummary_df.write.format("delta").save(cls.ordersummary_path)

cls.spark.sql(f"CREATE TABLE IF NOT EXISTS {cls.catalog}.{cls.schema}.customer USING DELTA LOCATION '{cls.customer_path}'")
cls.spark.sql(f"CREATE TABLE IF NOT EXISTS {cls.catalog}.{cls.schema}.ordersummary USING DELTA LOCATION '{cls.ordersummary_path}'")

```

Step 4: Updating the Customer Table and Running the SCD Type 2 Update

This step updates the customer table with the new data and runs the `update\_scd\_type2` function to update the order summary table.

```

self.customer_df_updated.write.format("delta").mode("overwrite").save(self.customer_path)

update_scd_type2(self.spark, self.catalog, self.schema)

```

Step 5: Verifying the Updates in the Order Summary Table

This step checks the order summary table for the expected changes after the SCD Type 2 update.

```

updated_ordersummary = self.spark.read.format("delta").load(self.ordersummary_path)

active_records = updated_ordersummary.filter("IsActive = true").count()
self.assertEqual(active_records, 3)

john_records = updated_ordersummary.filter("CustId = 'C001'").count()
self.assertEqual(john_records, 2)

john_active = updated_ordersummary.filter("CustId = 'C001' AND IsActive = true").first()
self.assertEqual(john_active["Name"], "John Doe Updated")

```

```
jane_records = updated_ordersummary.filter("CustId = 'C002'").count()  
self.assertEqual(jane_records, 2)  
  
jane_active = updated_ordersummary.filter("CustId = 'C002' AND IsActive = true").first()  
self.assertEqual(jane_active["Region"], "West")  
  
bob_records = updated_ordersummary.filter("CustId = 'C003'").count()  
self.assertEqual(bob_records, 1)
```